

EE5203 L04 Practice Worksheet

Pointers, bits, masks, and CMSIS registers

Basri Erdogan

Expected time: 30-45 minutes **Policy:** Attempt every question before consulting the worked solutions. All register questions can — and should — be answered on paper first.

Part A - Pointers and structures

- Trace by hand and give every final value:

```
/* (a) */
int a = 3;
int *q = &a;

*q = *q * 4;
a = a + 1;

/* (b) */
int m = 10;
int n = 20;
int *p = &m;

*p = *p + n;
p = &n;
*p = 0;
```

- `struct Point { int x; int y; };` and `struct Point pt = {4, 7};` `struct Point *pp = &pt;` — for each expression, state its value or say it is invalid:
 - `pt.x`
 - `pp->y`
 - `(*pp).x`
 - `*pp.y`
- Explain, in one sentence each, why the device header defines `GPIOA` as a **pointer** to a structure rather than a structure variable, and what `volatile` on each member guarantees.

Part B - Binary and hexadecimal

- Convert:
 - `0x0C00` to binary (16 bits) — which bits are set?
 - `0x00200000` to the single set bit's index.
 - binary `0000 0100 0001 0001` to hex.
 - `1U << 13` to hex.
- `GPIOA`'s `MODER` reads `0xA8000400`. Which pins are **not** in mode 00, and what is the mode of PA5? (Pin `n` owns bits `2n+1:2n`.)

Part C - Masks

- Write one statement each (operate on `uint32_t value`):
 - set bit 12;
 - clear bit 3;
 - toggle bit 0;
 - test whether bit 9 is set (as an `if` condition);
 - clear the two-bit field at bits 7:6, then write pattern 10 into it.
- `value` starts as `0x0000 00F0`. Give the value after **each** statement, applied in order:

```
value |= (1U << 8);
value &= ~(1U << 5);
value ^= (1U << 4);
value &= ~(3U << 2);
```

8. A student wants PA3 (mode bits 7:6) as output and writes only `GPIOA->MODER |= (1U << 6);`
 - a. For which starting field values does this produce the wrong mode?
 - b. Write the correct two-statement sequence.
9. Explain the difference between `if (flags & (1U << 4))` and `if (flags && (1U << 4))` — what does each test, and when do they give different answers?

Part D - CMSIS registers and GPIO

10. Decode `RCC->AHB1ENR |= (1U << 0);` word by word: `RCC`, `->`, `AHB1ENR`, `|=`, `1U << 0`. Why is `|=` essential here rather than `=`?
11. The homework wires an LED to PB0 and a button to PC1. Give the exact statements to:
 - a. enable the clocks for `GPIOB` **and** `GPIOC` in one statement;
 - b. make PB0 an output (field bits 1:0);
 - c. make PC1 an input with pull-up (`MODER` bits 3:2 = 00, `PUPDR` bits 3:2 = 01);
 - d. test in an `if` whether PC1 currently reads low (button pressed), using `GPIOC->IDR`.
12. Why is plain assignment (`=`) correct for `GPIOA->BSRR` but destructive for `GPIOA->MODER`? Name the register property that makes the difference.
13. Write the `BSRR` statements that drive PA5 high and low, and state the general rule for the reset bit of pin `n`.

Part E - Predict the register

14. After generated initialization, `GPIOA->MODER` reads `0xA8000400`. The lab's Checkpoint B/D sequence then executes:

```
GPIOA->MODER &= ~(3U << 10); /* observed value? */
GPIOA->MODER |= (1U << 10); /* observed value? */
```

Give both intermediate hex values.

15. In the SFR view you observe `RCC->AHB1ENR = 0x00000085`. Which GPIO ports are clocked? (`AHB1ENR` bit `n` clocks GPIO port `n`: A=0, B=1, C=2, ... H=7.)

Part F - Evidence

16. The LED blinks, but you must prove *“PA5 is configured as an output by my two MODER statements, not by leftover generated init.”* Describe a breakpoint plan and the two SFR observations that prove it.
17. Build is clean, LED never lights. Give three distinct register-level explanations and the single SFR observation that eliminates each.

Exit self-assessment

Mark one:

- ☐ Ready for the L04 lab without additional review
- ☐ Need to review pointers and `->`
- ☐ Need to review binary/hex and shifts
- ☐ Need to review the mask recipes
- ☐ Need to review the `RCC`/`MODER`/`BSRR` roles